TASK 18:ASSIGNMENT NO 18

Name of Student : Anuja Vilas Wankhade

Domain : Artificial intelligence and machine Learning (AIML)

College Name: Manav School of Polytechnic AKOLA

Branch : Computer Engineering

GitHub Repositories Link: https://github.com/anujaw193-star/python-project-collection

---

1. (Data Loading & Initial Analysis)

Code :

```
[3]
✓ 0s
# Load the Ford Car Dataset
df = pd.read_csv("ford_car_dataset - ford_car_dataset.csv")
# Display the first 10 rows
print("First 10 Rows:")
print(df.head(10))
# Display the last 5 rows
print("\nLast 5 Rows:")
print(df.tail())
# Check the shape of the dataset
print("\nShape of the Dataset:")
print(df.shape)
# Display data types of all columns
print("\nData Types of Columns:")
print(df.dtypes)
# Additional information about the dataset
print("\nDataset Information:")
df.info()
```

Output:

```
...  0  Fiesta  2017  12000    Automatic    15944   Petrol  150  57.7         1.0
     1   Focus  2018  14000       Manual     9083   Petrol  150  57.7         1.0
     2   Focus  2017  13000       Manual    12456   Petrol  150  57.7         1.0
     3  Fiesta  2019  17500       Manual    10460   Petrol  145  40.3         1.5
     4  Fiesta  2019  16500    Automatic     1482   Petrol  145  48.7         1.0
     5  Fiesta  2015  10500       Manual    35432   Petrol  145  47.9         1.6
     6    Puma  2019  22500       Manual     2029   Petrol  145  50.4         1.0
     7  Fiesta  2017   9000       Manual    13054   Petrol  145  54.3         1.2
     8    Kuga  2019  25500    Automatic     6894   Diesel  145  42.2         2.0
     9   Focus  2018  10000       Manual    48141   Petrol  145  61.4         1.0

     Last 5 Rows:
           model  year  price transmission  mileage fuelType  tax   mpg  \
     17961  B-MAX  2017   8999       Manual    16700   Petrol  150  47.1
     17962  B-MAX  2014   7499       Manual    40700   Petrol   30  57.7
     17963  Focus  2015   9999       Manual     7010   Diesel   20  67.3
     17964     KA  2018   8299       Manual     5007   Petrol  145  57.7
     17965  Focus  2015   8299       Manual     5007   Petrol   22  57.7
```

```
       engineSize
17961         1.4
17962         1.0
17963         1.6
17964         1.2
17965         1.0

Shape of the Dataset:
(17966, 9)

Data Types of Columns:
model            object
year              int64
price             int64
transmission     object
mileage           int64
fuelType         object
tax               int64
mpg             float64
engineSize      float64
dtype: object
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 17966 entries, 0 to 17965
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   model         17966 non-null  object
 1   year          17966 non-null  int64
 2   price         17966 non-null  int64
 3   transmission  17966 non-null  object
 4   mileage       17966 non-null  int64
 5   fuelType      17966 non-null  object
 6   tax           17966 non-null  int64
 7   mpg           17966 non-null  float64
 8   engineSize    17966 non-null  float64
dtypes: float64(2), int64(4), object(3)
memory usage: 1.2+ MB
```

 # Observations:

 # 1. The dataset contains multiple rows representing different Ford car records.

 # 2. Each row corresponds to one car and each column represents a feature of the car.

 # 3. The shape of the dataset tells us the total number of rows (records) and columns (features).

 # 4. The dataset contains both numerical and categorical data types.

 # 5. Numerical columns may include features like year, price, mileage, engine size, etc.

 # 6. Categorical columns may include features like model and transmission.

 # 7. The dataset structure appears suitable for data analysis and machine learning after checking for missing values and preprocessing.

---

2. (Missing & Duplicate Values)

   Code:

```
[4]
✓ 0s
# Check missing values in each column
print("Missing Values in Each Column:")
print(df.isnull().sum())

# Total missing values in the dataset
print("\nTotal Missing Values in the Dataset:")
print(df.isnull().sum().sum())

# Check for duplicate rows
duplicate_rows = df.duplicated().sum()
print("\nNumber of Duplicate Rows:", duplicate_rows)

# Remove duplicate rows
df = df.drop_duplicates()

# Verify duplicates after removal
print("\nDuplicate Rows After Removal:")
print(df.duplicated().sum())

# Display the new shape of the dataset
print("\nDataset Shape After Removing Duplicates:")
print(df.shape)
```

Output:

```
Missing Values in Each Column:
model           0
year            0
price           0
transmission    0
mileage         0
fuelType        0
tax             0
mpg             0
engineSize      0
dtype: int64

Total Missing Values in the Dataset:
0

Number of Duplicate Rows: 154

Duplicate Rows After Removal:
0

Dataset Shape After Removing Duplicates:
(17812, 9)
```

 # Summary:

 # 1. Checked all columns for missing (null) values using df.isnull().sum().

 # 2. Calculated the total number of missing values in the dataset.

 # 3. Identified duplicate rows using df.duplicated().sum().

 # 4. Removed duplicate rows using df.drop_duplicates().

 # 5. Verified that no duplicate rows remain after cleaning.

 # 6. This cleaning process improves data quality and ensures more accurate analysis and machine learning results.

---

3. *(Statistical Summary)*

   Statistical summary of Numeric Columns

   Code:

```
[5]
✓ 0s
# Generate statistical summary
print("Statistical Summary of Numeric Columns:")
print(df.describe())
```

Output:

```
Statistical Summary of Numeric Columns:
                year         price       mileage           tax           mpg  \
count  17812.000000  17812.000000  17812.000000  17812.000000  17812.000000
mean    2016.862396  12269.556310  23381.146362    113.315012     57.908696
std        2.052039   4736.285417  19419.011045     62.034603     10.132696
min     1996.000000    495.000000      1.000000      0.000000     20.800000
25%     2016.000000   8999.000000  10000.000000     30.000000     52.300000
50%     2017.000000  11288.000000  18277.000000    145.000000     58.900000
75%     2018.000000  15295.000000  31098.500000    145.000000     65.700000
max     2060.000000  54995.000000  177644.000000   580.000000    201.800000

         engineSize
count  17812.000000
mean       1.350623
std        0.432581
min        0.000000
25%        1.000000
50%        1.200000
75%        1.500000
max        5.000000
```

Minimum, Maximum, Mean, and Median of Key Columns

Code:

```
[6]
✓ 0s
# List of key columns
key_columns = ['price', 'mileage', 'year']

for col in key_columns:
  print(f"\n-----{col.upper()}-----")
  print("Minimum :", df[col].min())
  print("Maximum :", df[col].max())
  print("Mean :", df[col].mean())
  print("Median :", df[col].median())
  print("Standard Deviation :", df[col].std())
```

```
-----PRICE-----
Minimum : 495
Maximum : 54995
Mean : 12269.556310352571
Median : 11288.0
Standard Deviation : 4736.2854173588985

-----MILEAGE-----
Minimum : 1
Maximum : 177644
Mean : 23381.146362003143
Median : 18277.0
Standard Deviation : 19419.011044627077

-----YEAR-----
Minimum : 1996
Maximum : 2060
Mean : 2016.8623961374353
Median : 2017.0
Standard Deviation : 2.0520394542458527
```

Alternative (Tabular Format)

```
[7]
✓ 0s
summary = pd.DataFrame({
    "Minimum": df[key_columns].min(),
    "Maximum": df[key_columns].max(),
    "Mean": df[key_columns].mean(),
    "Median": df[key_columns].median(),
    "Standard Deviation": df[key_columns].std()
})

print("Summary Statistics for Key Columns:")
print(summary)

Summary Statistics for Key Columns:
           Minimum  Maximum          Mean  Median  Standard Deviation
price          495    54995  12269.556310 11288.0         4736.285417
mileage          1   177644  23381.146362 18277.0        19419.011045
year          1996     2060   2016.862396  2017.0            2.052039
+ Code  + Text
```

Summary (Comments/Markdown)

 # Summary:

 # 1. Used df.describe() to generate descriptive statistics for all numeric columns.

 # 2. The summary includes count, mean, standard deviation, minimum, 25%, 50%, 75%, and maximum values.

 # 3. Calculated the minimum, maximum, mean, and median for the key columns:

 # - price

 # - mileage

 # - year

 # 4. These statistics help understand the distribution, central tendency,

 # and range of the dataset, making it easier to identify patterns and outliers.

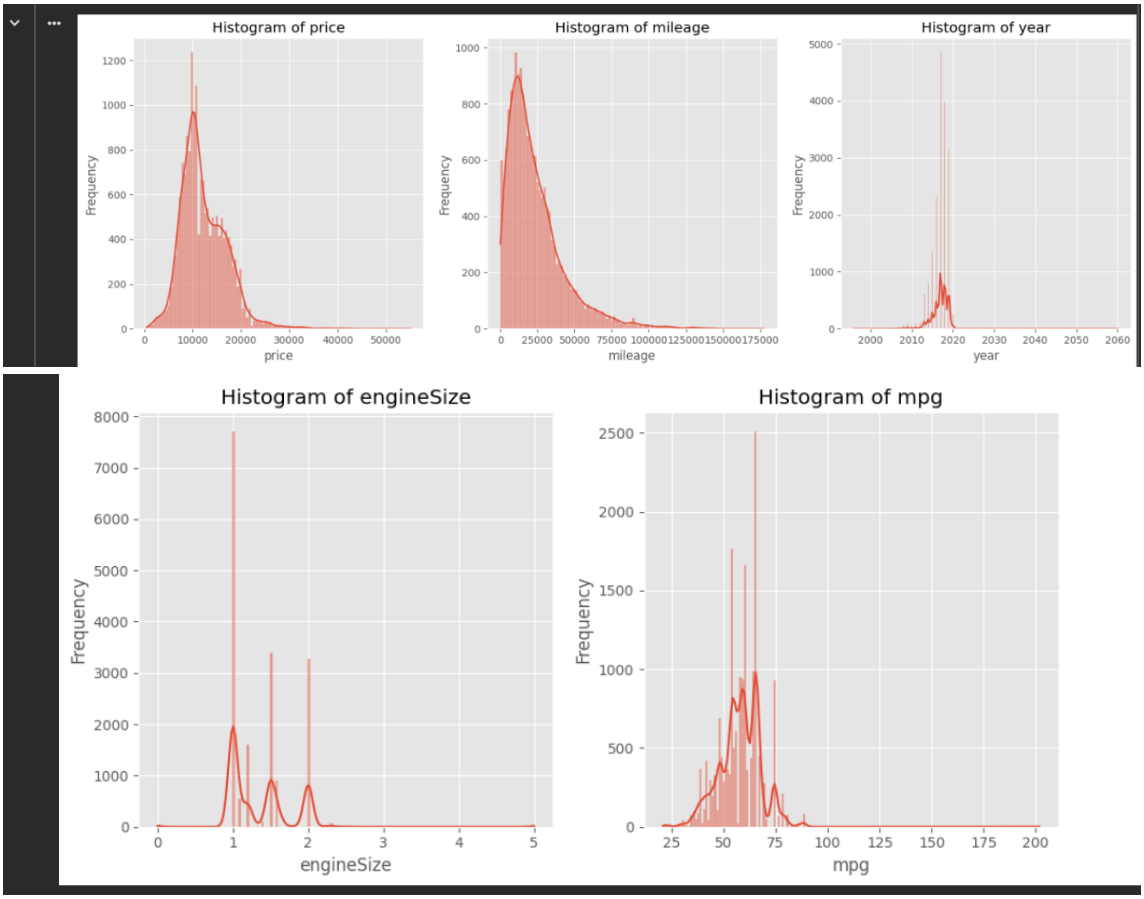
---

4. *(Histogram of Numeric Features)*

Code:

```
[11]
✓ 4s
# List of important numeric columns
numeric_columns = ['price', 'mileage', 'year', 'engineSize', 'mpg']

# Set figure size
plt.figure(figsize=(15, 10))

# Plot histogram for each column
for i, col in enumerate(numeric_columns, 1):
    plt.subplot(2, 3, i)
    sns.histplot(df[col], kde=True)
    plt.title(f'Histogram of {col}')
    plt.xlabel(col)
    plt.ylabel('Frequency')

plt.tight_layout()
plt.show()
```

Output:



Summary:

 # Findings:

 # 1. Price:

 # - Shows how car prices are distributed.

 # - A right-skewed distribution indicates that most cars are lower-priced,

 # with a few expensive cars.

 # 2. Mileage:

 # - Displays the distribution of mileage values.

 # - Higher mileage vehicles are generally older and more used.

 # 3. Year:

 # - Indicates the manufacturing year of the cars.

 # - More recent years usually have a higher number of cars in the dataset.

 # 4. Engine Size:

 # - Most cars have small to medium engine sizes.

 # - Large engine sizes occur less frequently.

 # 5. MPG (Miles Per Gallon):

 # - Shows the fuel efficiency distribution.

 # - Most cars are expected to have moderate MPG values,

 # while very low or very high MPG values are less common.

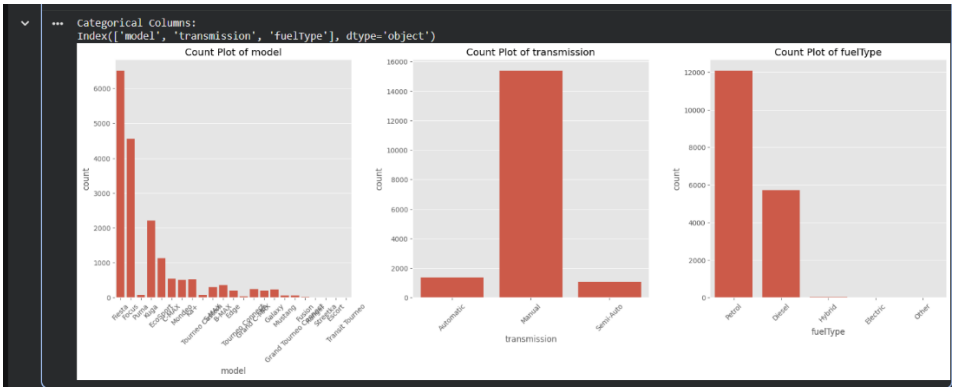
---

5. *(Count Plots of Categorical Features)*

Code:

```
[12]
✓ 0s
# Find all categorical columns
categorical_columns = df.select_dtypes(include=['object']).columns

print("Categorical Columns:")
print(categorical_columns)

# Set figure size
plt.figure(figsize=(18,12))

# Plot count plots
for i, col in enumerate(categorical_columns, 1):
    plt.subplot(2, 3, i)
    sns.countplot(data=df, x=col)
    plt.title(f'Count Plot of {col}')
    plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```

Output:



Summary:

 # Insights:

 # 1. Count plots were created for all categorical columns such as

 # model, transmission, fuelType, etc.

#

 # 2. The category with the highest count in each plot represents

 # the most common type in the dataset.

#

 # 3. The most common fuel type indicates the preferred fuel choice

 # among Ford car buyers.

#

 # 4. The most frequent transmission type shows whether manual or

 # automatic cars are more popular.

#

 # 5. The model count plot highlights the best-selling Ford models

 # included in the dataset.

#

 # 6. These plots help understand customer preferences and overall
 # market trends for Ford vehicles.

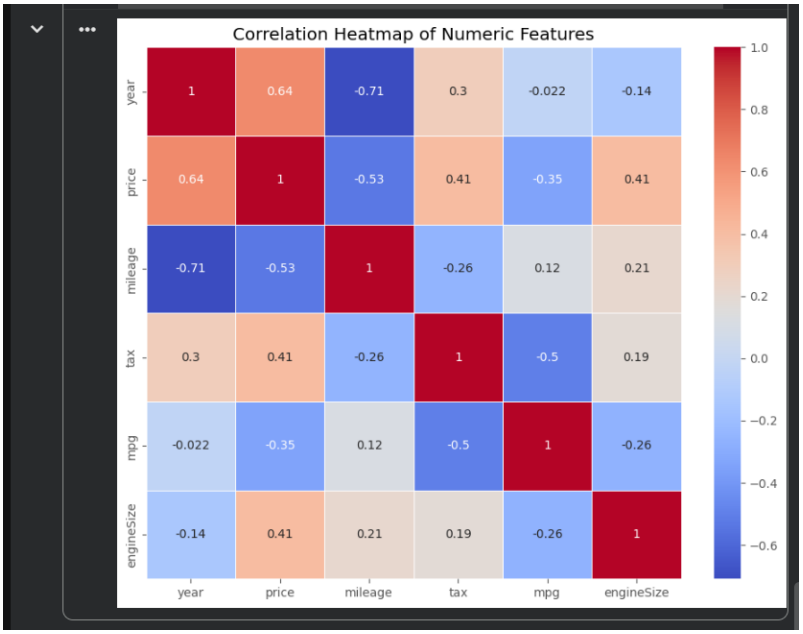
---

6. *(Correlation Heatmap)*

Code:

```
[13]
✓ 0s
# Select only numeric columns
numeric_df = df.select_dtypes(include=['int64', 'float64'])
# Correlation matrix
corr_martrix = numeric_df.corr()
# Plot heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(corr_martrix, annot=True, cmap='coolwarm', linewidths=0.5)
plt.title("Correlation Heatmap of Numeric Features")
plt.show()
```

Output:



Summary:

 # Observations:
 # 1. The heatmap shows the correlation between all numeric features.
 # 2. Correlation values range from -1 to +1.
 # - +1 indicates a strong positive correlation.
 # - -1 indicates a strong negative correlation.
 # - 0 indicates little or no correlation.
#
 # 3. Features with higher positive correlation with 'price'
 # are more likely to influence the car price.
#
 # 4. Features with negative correlation indicate that as the
 # feature value increases, the price generally decreases.
#
 # 5. Based on the correlation values printed above, identify

 # which features have the strongest positive and negative
 # relationship with 'price'.

---

7. *(Feature Identification)*

Code:

```
[14]
✓ 0s
# Independent Featurs (Input Features)
X = df.drop("price", axis=1)
# Dependent Featurs (Target Features)
y = df["price"]
print("Independent Features:")
print(X.columns.tolist())
print("\nDependent Features:")
print(y.name)
```

Output:

```
Independent Features:
['model', 'year', 'transmission', 'mileage', 'fuelType', 'tax', 'mpg', 'engineSize']

Dependent Features:
price
```

Summary:

 # Feature Identification

 # Independent Features (Input Features):
 # - model
 # - year
 # - transmission
 # - mileage
 # - fuelType
 # - tax
 # - mpg
 # - engineSize
#
 # (If your dataset contains any additional columns except 'price',
 # they are also considered independent features.)

 # Dependent Feature (Output/Target Feature):
 # - price

 # Justification:
 # 1. The independent features are the variables used to predict the value of the car.
 # 2. The dependent feature is 'price' because it is the output we want to predict.
 # 3. Features such as year, mileage, engine size, fuel type, and transmission
 # directly influence the selling price of a car.

 # 4. Therefore, this is a Supervised Machine Learning Regression problem,
 # since the target variable (price) is continuous.

---

8. *(Encoding Categorical Variables)*

Show Categorical Columns

Code & Output:

```
[15]
✓ 0s
# Display all categorical columns
categorical_columns = df.select_dtypes(include=['object']).columns
print("Categorical Columns:")
print(categorical_columns)

Categorical Columns:
Index(['model', 'transmission', 'fuelType'], dtype='object')
+ Code  + Text
```

Show Before Encoding (First 5 Rows of 2 Columns)

Code & Output:

```
[16]
✓ 0s
# Before Encoding
print("Befor One-Hot Encoding:")
print(df[['model', 'fuelType']].head())

Befor One-Hot Encoding:
    model fuelType
0  Fiesta   Petrol
1   Focus   Petrol
2   Focus   Petrol
3  Fiesta   Petrol
4  Fiesta   Petrol
```

Apply One-Hot Encoding

Code & Output:

```
[17]
✓ 0s
# Apply One-Hot Encoding
df_encoded = pd.get_dummies(df, columns=categorical_columns, drop_first=True)
print("Encoding Complete Successfully!")

Encoding Complete Successfully!
```

Show After Encoding

Code:

```
[18]
✓ 0s
# After Encoding
print("First 5 Rows After Encoding:")
print(df_encoded.head())
print("\nShape of the Encoded Dataset:", df.shape)
print("Shape After Encoding:", df_encoded.shape)
```

Output:

```
First 5 Rows After Encoding:
   year  price  mileage  tax   mpg  engineSize  model_C-MAX  model_EcoSport  \
0  2017  12000    15944  150  57.7         1.0        False           False
1  2018  14000     9083  150  57.7         1.0        False           False
2  2017  13000    12456  150  57.7         1.0        False           False
3  2019  17500    10460  145  40.3         1.5        False           False
4  2019  16500     1482  145  48.7         1.0        False           False

   model_Edge  model_Escort  ...  model_Streetka  model_Tourneo Connect  \
0       False         False  ...           False                  False
1       False         False  ...           False                  False
2       False         False  ...           False                  False
3       False         False  ...           False                  False
4       False         False  ...           False                  False

   model_Tourneo Custom  model_Transit Tourneo  transmission_Manual  \
0                 False                  False                False
1                 False                  False                 True
2                 False                  False                 True
3                 False                  False                 True
4                 False                  False                False
```

```
   model_Tourneo Custom  model_Transit Tourneo  transmission_Manual  \
0                 False                  False                False
1                 False                  False                 True
2                 False                  False                 True
3                 False                  False                 True
4                 False                  False                False

   transmission_Semi-Auto  fuelType_Electric  fuelType_Hybrid  fuelType_Other  \
0                   False              False            False           False
1                   False              False            False           False
2                   False              False            False           False
3                   False              False            False           False
4                   False              False            False           False

   fuelType_Petrol
0             True
1             True
2             True
3             True
4             True

[5 rows x 34 columns]

Shape of the Encoded Dataset: (17812, 9)
Shape After Encoding: (17812, 34)
```

Display Newly Created Columns

Code & Output:

```
[19]
0s
# Display encoded columns for model and fuelType
print("Encoded Columns:")
print([col for col in df_encoded.columns if 'model_' in col or 'fuelType_' in col])

Encoded Columns:
['model_C-MAX', 'model_EcoSport', 'model_Edge', 'model_Escort', 'model_Fiesta', 'model_Focus', 'model_Fusion', 'model_Galaxy', 'model_Grand
```

Summary:

 # 1. Identified all categorical columns using select_dtypes().

 # 2. Displayed the original values of 'model' and 'fuelType'

 # before encoding.

 # 3. Applied One-Hot Encoding using pd.get_dummies().

 # 4. Converted all categorical variables into numeric columns.

 # 5. Used drop_first=True to avoid the Dummy Variable Trap.

 # 6. Displayed the dataset after encoding and compared its shape.

---

9. *(Feature Scaling)*

Code:

```
[27]
✓ 0s
# Independent Features
X = df_encoded.drop('price', axis=1)
# Dependent Feture
y = df_encoded['price']
# Independent StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Convert scaled array into DataFrame
X_scaled = pd.DataFrame(X_scaled, columns=X.columns)
# Display first 5 rows
print("First 5 Rows of Scaled Data:")
print(X_scaled.head())
```

Output:

```
First 5 Rows of Scaled Data:
       year   mileage       tax       mpg  engineSize  model_C-MAX  \
0  0.067059 -0.382994  0.591380 -0.020597   -0.810561    -0.177155
1  0.554393 -0.736317  0.591380 -0.020597   -0.810561    -0.177155
2  0.067059 -0.562616  0.591380 -0.020597   -0.810561    -0.177155
3  1.041726 -0.665405  0.510777 -1.737858    0.345325    -0.177155
4  1.041726 -1.127749  0.510777 -0.908836   -0.810561    -0.177155

   model_EcoSport  model_Edge  model_Escort  model_Fiesta  ...  \
0       -0.259896   -0.107903     -0.007493      1.317770  ...
1       -0.259896   -0.107903     -0.007493     -0.758858  ...
2       -0.259896   -0.107903     -0.007493     -0.758858  ...
3       -0.259896   -0.107903     -0.007493      1.317770  ...
4       -0.259896   -0.107903     -0.007493      1.317770  ...

   model_Streetka  model_Tourneo Connect  model_Tourneo Custom  \
0       -0.010597              -0.042424             -0.062361
1       -0.010597              -0.042424             -0.062361
2       -0.010597              -0.042424             -0.062361
3       -0.010597              -0.042424             -0.062361
4       -0.010597              -0.042424             -0.062361

   model_Transit Tourneo  transmission_Manual  transmission_Semi-Auto  \
0              -0.007493            -2.516557               -0.253434
1              -0.007493             0.397368               -0.253434
2              -0.007493             0.397368               -0.253434
3              -0.007493             0.397368               -0.253434
4              -0.007493            -2.516557               -0.253434

   fuelType_Electric  fuelType_Hybrid  fuelType_Other  fuelType_Petrol
0          -0.010597        -0.035166       -0.007493         0.688753
1          -0.010597        -0.035166       -0.007493         0.688753
2          -0.010597        -0.035166       -0.007493         0.688753
3          -0.010597        -0.035166       -0.007493         0.688753
4          -0.010597        -0.035166       -0.007493         0.688753

[5 rows x 33 columns]
```

Summary:

 # 1. Separated the independent features (X) and target feature (price).

 # 2. Used StandardScaler from sklearn.preprocessing.

 # 3. Standardized all independent numeric features.

 # 4. After scaling, each feature has approximately:

 # - Mean = 0

 # - Standard Deviation = 1

 # 5. Displayed the first five rows of the scaled dataset.

 # 6. Feature scaling improves the performance of many machine learning algorithms.

10. (Mini Project – Complete Preprocessing Pipeline)

Code:

[28] ✓ 3s

```
# ================================================
# MINI PROJECT - COMPLETE PREPROCESSING PIPELINE
#================================================

# 1. Load the Dataset
df = pd.read_csv("ford_car_dataset - ford_car_dataset.csv")

print("First 5 Rows:")
print(df.head())

print("\nDataset Shape:", df.shape)

# ================================================
# Handle Missing Values
# ================================================

print("\nMissing Values:")
print(df.isnull().sum())

# Remove missing values
df = df.dropna()
```

```
# ================================================
# Handle Duplication Values
# ================================================

print("\nDuplicate Rows:", df.duplicated().sum())

df = df.drop_duplicates()

print("\nDataset Shape After Cleaning:", df.shape)

# ================================================
# 2. Exploratory Data Analysis (EDA)
# ================================================

# Historgrams
numeric_columns = ['price', 'mileage', 'year', 'engineSize', 'mpg']

plt.figure(figsize=(15, 10))
for i, col in enumerate(numeric_columns, 1):
    plt.subplot(2, 3, i)
    sns.histplot(df[col], kde=True)
    plt.title(col)
    plt.xlabel(col)

plt.tight_layout()
plt.show()
```

[28] ✓ 3s

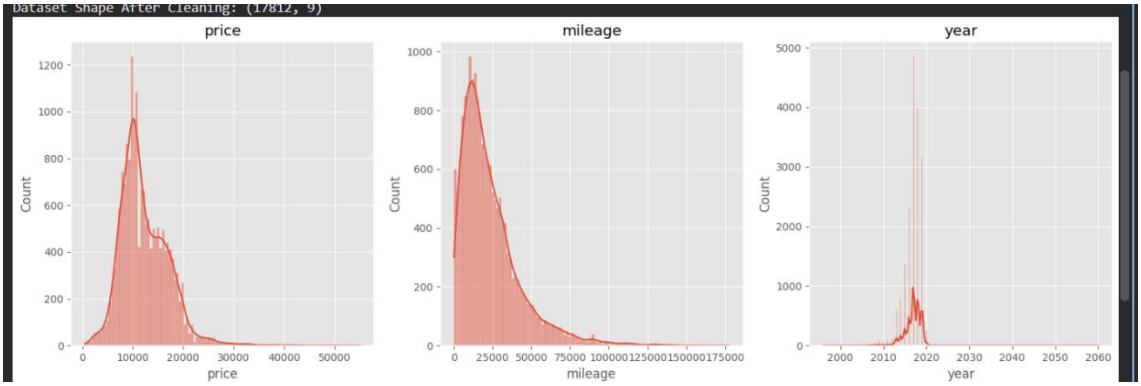
```
# Count Plots
categorical_columns = df.select_dtypes(include=['object']).columns
for col in categorical_columns:
    plt.figure(figsize=(10,5))
    sns.countplot(data=df, x=col,
                  order=df[col].value_counts().index)
    plt.xticks(rotation=45)
    plt.title(col)
    plt.show()
# Correlation Heatmap
plt.figure(figsize=(10, 8))

corr = df.select_dtypes(include=['int64', 'float64']).corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()

# =====================================
# 3. Input and Output Features
# =====================================

X = df.drop('price', axis=1)
y = df['price']

print("Independent Features:")
print(X.columns)
print("\nTarget Feature:")
print(y.name)
```

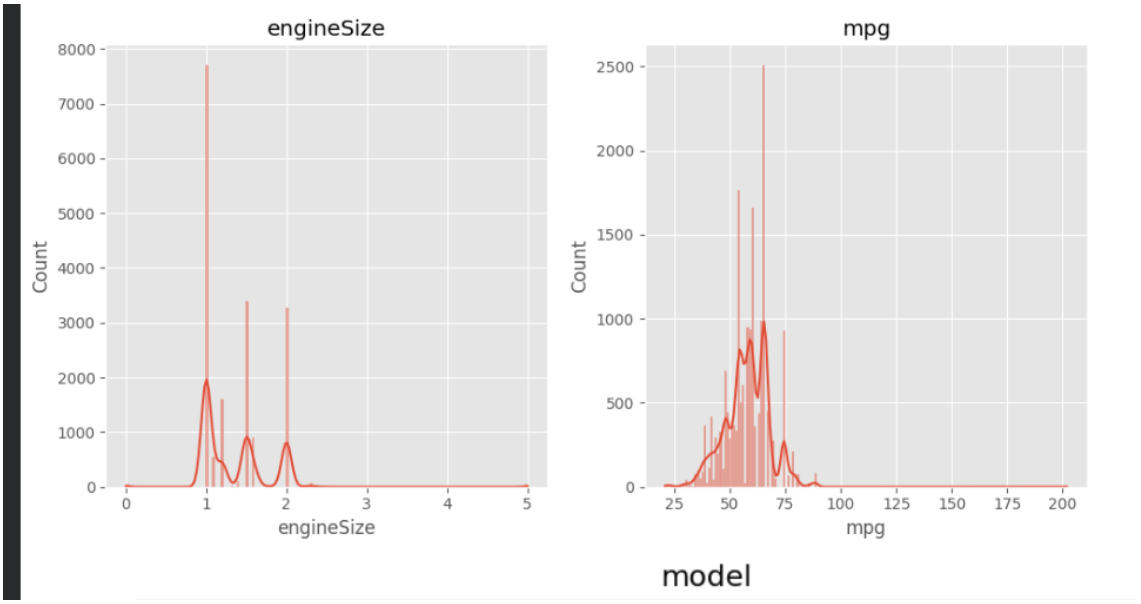
```
# ====================================
# 4. One-Hot Encoding
# ====================================

df_encoded = pd.get_dummies(df, columns=categorical_columns, drop_first=True)

print("\nShape Before Encoding:", df.shape)
print("Shape After Encoding:", df_encoded.shape)

# ====================================
# 5. Feature Scaling
# ====================================

X = df_encoded.drop('price', axis=1)
y = df_encoded['price']
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_scaled = pd.DataFrame(X_scaled, columns=X.columns)

print("\nFirst 5 Rows of Scaled Data:")
print(X_scaled.head())
```

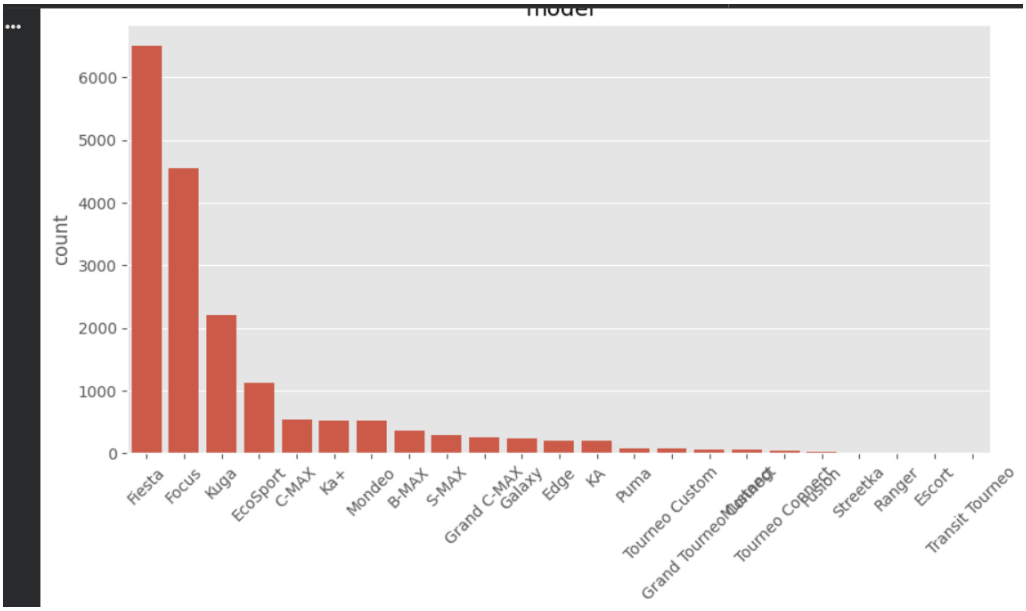
Output:

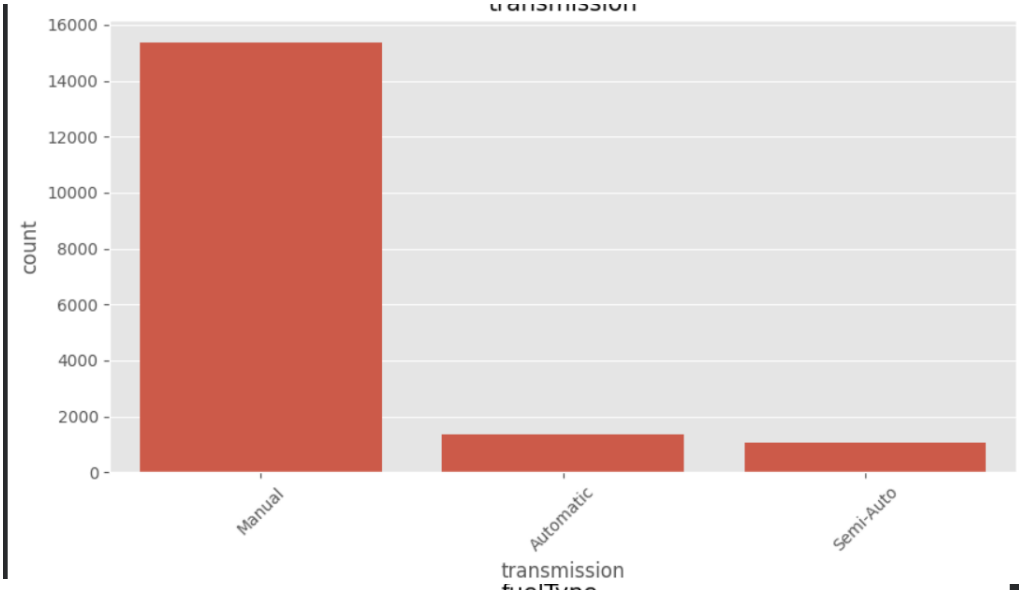
```
First 5 Rows:
     model  year  price transmission  mileage fuelType  tax   mpg  engineSize
0  Fiesta  2017  12000    Automatic    15944   Petrol  150  57.7         1.0
1   Focus  2018  14000       Manual     9083   Petrol  150  57.7         1.0
2   Focus  2017  13000       Manual    12456   Petrol  150  57.7         1.0
3  Fiesta  2019  17500       Manual    10460   Petrol  145  40.3         1.5
4  Fiesta  2019  16500    Automatic     1482   Petrol  145  48.7         1.0

Dataset Shape: (17966, 9)

Missing Values:
model           0
year            0
price           0
transmission    0
mileage         0
fuelType        0
tax             0
mpg             0
engineSize      0
dtype: int64

Duplicate Rows: 154
```

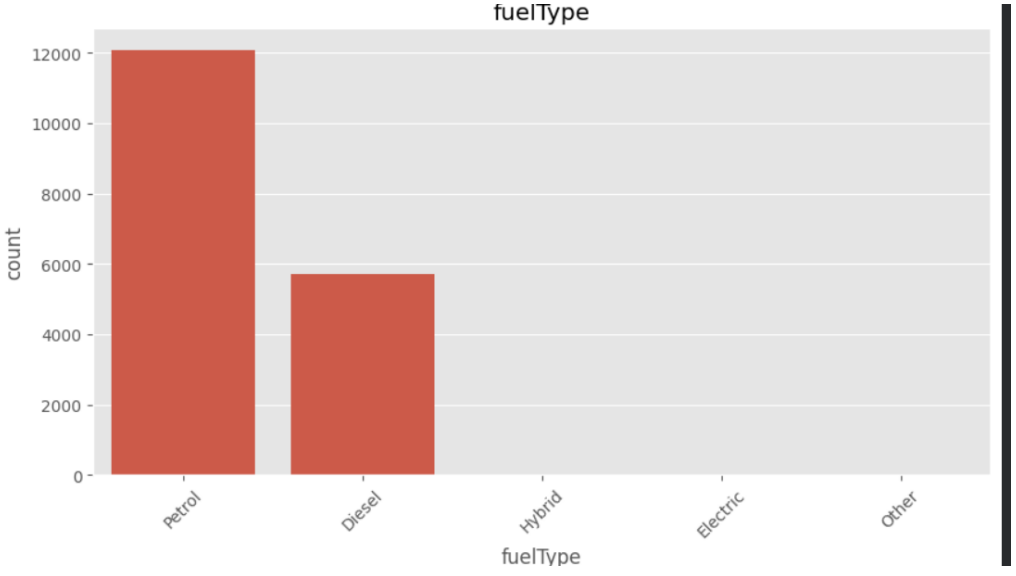
```
Dataset Shape After Cleaning: (17812, 9)
```

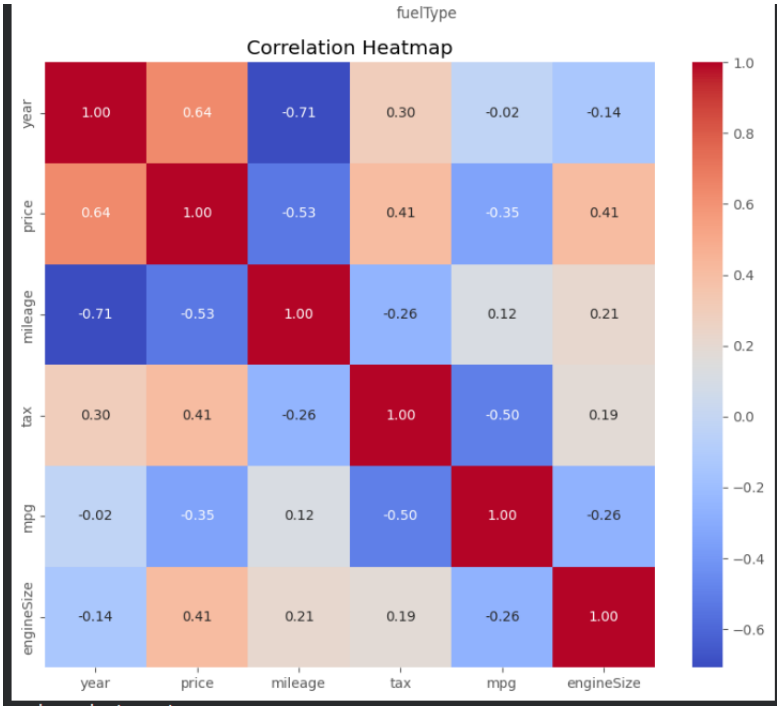





model
count
6000
5000
4000
3000
2000
1000
0
Fiesta
Focus
Kuga
EcoSport
C-MAX
Ka+
Mondeo
B-MAX
S-MAX
Grand C-MAX
Galaxy
Edge
KA
Puma
Tourneo Custom
Tourneo Connect
Fusion
Streetka
Ranger
Escort
Transit Tourneo


transmission
count
16000
14000
12000
10000
8000
6000
4000
2000
0
Manual
Automatic
Semi-Auto
transmission


fuelType
count
12000
10000
8000
6000
4000
2000
0
Petrol
Diesel
Hybrid
Electric
Other
fuelType




```
Independent Features:
Index(['model', 'year', 'transmission', 'mileage', 'fuelType', 'tax', 'mpg',
       'engineSize'],
      dtype='object')

Target Feature:
price

Shape Before Encoding: (17812, 9)
Shape After Encoding: (17812, 34)

First 5 Rows of Scaled Data:
       year   mileage       tax       mpg  engineSize  model_C-MAX  \
0  0.067059 -0.382994  0.591380 -0.020597   -0.810561    -0.177155
1  0.554393 -0.736317  0.591380 -0.020597   -0.810561    -0.177155
2  0.067059 -0.562616  0.591380 -0.020597   -0.810561    -0.177155
3  1.041726 -0.665405  0.510777 -1.737858    0.345325    -0.177155
4  1.041726 -1.127749  0.510777 -0.908836   -0.810561    -0.177155

   model_EcoSport  model_Edge  model_Escort  model_Fiesta  ...  \
0       -0.259896   -0.107903     -0.007493      1.317770  ...
1       -0.259896   -0.107903     -0.007493     -0.758858  ...
2       -0.259896   -0.107903     -0.007493     -0.758858  ...
3       -0.259896   -0.107903     -0.007493      1.317770  ...
4       -0.259896   -0.107903     -0.007493      1.317770  ...
```

```
   model_Streetka  model_Tourneo Connect  model_Tourneo Custom  \
0       -0.010597              -0.042424             -0.062361
1       -0.010597              -0.042424             -0.062361
2       -0.010597              -0.042424             -0.062361
3       -0.010597              -0.042424             -0.062361
4       -0.010597              -0.042424             -0.062361

   model_Transit Tourneo  transmission_Manual  transmission_Semi-Auto  \
0              -0.007493            -2.516557               -0.253434
1              -0.007493             0.397368               -0.253434
2              -0.007493             0.397368               -0.253434
3              -0.007493             0.397368               -0.253434
4              -0.007493            -2.516557               -0.253434

   fuelType_Electric  fuelType_Hybrid  fuelType_Other  fuelType_Petrol
0          -0.010597        -0.035166       -0.007493         0.688753
1          -0.010597        -0.035166       -0.007493         0.688753
2          -0.010597        -0.035166       -0.007493         0.688753
3          -0.010597        -0.035166       -0.007493         0.688753
4          -0.010597        -0.035166       -0.007493         0.688753

[5 rows x 33 columns]
```